

RL on Custom Junimo Kart Environment

Abstract

This project applies reinforcement learning methods, those being Proximal Policy Optimization (PPO) and Double Deep Q-Networks (DDQN), on a custom Gymnasium environment inspired by the *Junimo Kart* mini game from *Stardew Valley*. I designed a procedural tile scrolling platformer environment, with a small, discrete action space (whether to jump or not), internal physics, obstacles, and its sparse terminal rewards. I evaluated the performance of both RL algorithms, discovering that on this platformer environment, DDQN performance showed improved learning and environment understanding in comparison to PPO.

I. INTRODUCTION

Reinforcement learning (RL) models are often introduced using standardized benchmark environments such as CartPole, MountainCar, or the projects explored in class (e.g. GridWorld, Frogger, and the simple hopper). However, applying them to more advanced, custom tasks may provide a deeper understanding of different behaviors and limitations of these algorithms. I hoped to design a fully custom Gymnasium environment inspired by the *Junimo Kart* mini game from *Stardew Valley*, creating a procedural side-scrolling platformer containing obstacles, collectibles, and pits.

The project focuses on exploring two methods studied in class: Proximal Policy Optimization (PPO) and Double Deep Q-Networks (DDQN). By training each algorithm on the new environment, we can see the relative strengths, as well as weaknesses, in exploration, learning stability, and behavioral consistency. Results seem to generally show that DDQN achieves more reliable learning and better long-term survival behavior compared to PPO in this particular environment, which is surprising, as we often see PPO outperform DDQN.

II. METHODOLOGY

A. Junimo Gymnasium Environment

To evaluate PPO and DDQN on a custom domain, I created a custom Gymnasium environment, which initially emulated the Junimo Kart minigame, but became its own environment due to my limited experience in game creation, which I am still satisfied with. The environment is a procedural side-scrolling platformer in which the player controls a character that continuously moves to the right, needing to avoid obstacles and collect coins and fruits for score.

Each observation contains two components, with the **Observation Space** being shown below:

- Tile Grid (5 x 14 x 20)
 - This grid is a 5-channel spatial representation of the local view of the map.

- The 5 channels correspond to ground tiles, roadblocks that bounce the player from above, boulders that allow the player to stand on them, coins, and fruits.
- State Vector (dimension 6)
 - This includes player vertical position, vertical velocity, grounded state, distance from the next pit, distance from the next fruit, and the next ground height.
 - Since the player stays in the third column, we do not need to track the player on the tile grid, batching this in with other variables into the non-convolutional space.

The action space is a simple, discrete binary choice. When attempting to jump, it is limited to the physics of the environment, as well as the `max_jump_hold` variable (7) that prevents jump impact after 7 frames of jumping, resetting when on the ground.

- 0 - Do nothing
- 1- Attempt to Jump

The environment simulates internal physics similar to a platformer. When jumping off the ground, our y velocity is set to 4.0, with a boost of 0.3 when in the air. Regardless, a gravity of -0.7 is applied at every frame, with the y velocity being multiplied by 0.3 before being added to our current y position to find the next one. If the player ends on the ground on the next frame, y velocity is reset to 0, allowing jumps to be independent from each other.

In terms of obstacle spawning conditions, ground spawning has 4 cases, which allow the game to create pits with different pit lengths and adhere to pit cooldowns. Fruit columns are determined beforehand, guaranteeing fruits are decently spaced out, with an additional condition to prevent obstacles within 6 tiles of the fruit (to make it collectible). Coins spawn randomly at a 10% rate, with a cooldown of 10 columns between coins. Obstacles spawn with 4.5% of ground boulders, 3% chance of roadblocks, and 10% chance of air boulders with a cooldown of 4 tiles. Note that no objects can spawn in the same column as each other, to prevent overwriting of spawns.

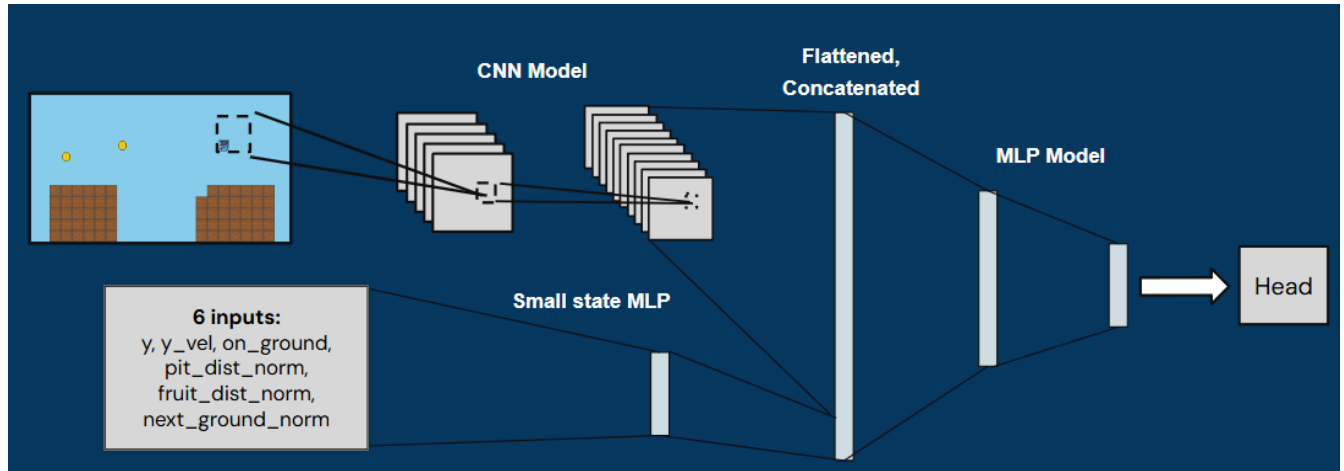
After reward tuning, I eventually set these values of my RL rewards to these values:

- Any Death (Pits and Obstacles): -4.5
- Coins: 3.0
- Fruits: 15.0
- Survival (every frame): 0.01
- Obstacle bounce/stand: 0.03
- Level complete (ideal but unlikely to occur): 100.0
- Pit survival: 0.5
- Jump: -0.002 (To prevent excessive jump spamming)

Note: The game score matches what we would see in JunimoKart; RL rewards were shaped to help the models learn the environment.

B. Proximal Policy Optimization (PPO) and Double Deep Q-Network (DDQN)

To implement PPO and DDQN in this new custom Gymnasium environment, I imported and adapted the code we wrote for Projects 2-4. Although the high-level training loop structures and fundamental PPO and DDQN remain similar, all the neural networks (policy, value, and Q-networks) required modification due to the complexity of the state inputs. This environment provides a state observation (5 x 14 x 20 tile grid) along with a 6-dimensional state vector. Supporting both inputs while keeping networks lightweight led to a hybrid architecture being implemented.



This figure shows the architecture shared by each network in both PPO and DDQN. The state observation grid is processed using a small convolutional encoder, resulting in a flattened single representation of the grid. Separately, the 6-dimensional non-spatial vector is passed through a small MLP before being concatenated and fed to a larger connected network. This is then processed into a head, which is the output that differs between each network. This results in the Policy logits for the policy network, a single advantage value for the Value network, and the Q-values for each action for the DDQN.

Due to the difficulty and sparsity of rewards in the environment, PPO and DDQN were trained with a curriculum schedule that adjusted the maximum episode length after a certain number of steps.

In PPO, we had parameters set to:

- 5,000,000 total steps
- 50,000 evaluation frequency
- 500,000 update schedule, with schedule: [250, 300, 400, 550, 700, 900, 1000, 1200] starting at 200.

In DDQN, our parameters were:

- 500,000 total steps (DDQN training is much slower than PPO per step)
- 5,000 evaluation frequency
- 100,000 update schedule, with schedule: [300, 550, 9000, 1200] starting at 200.

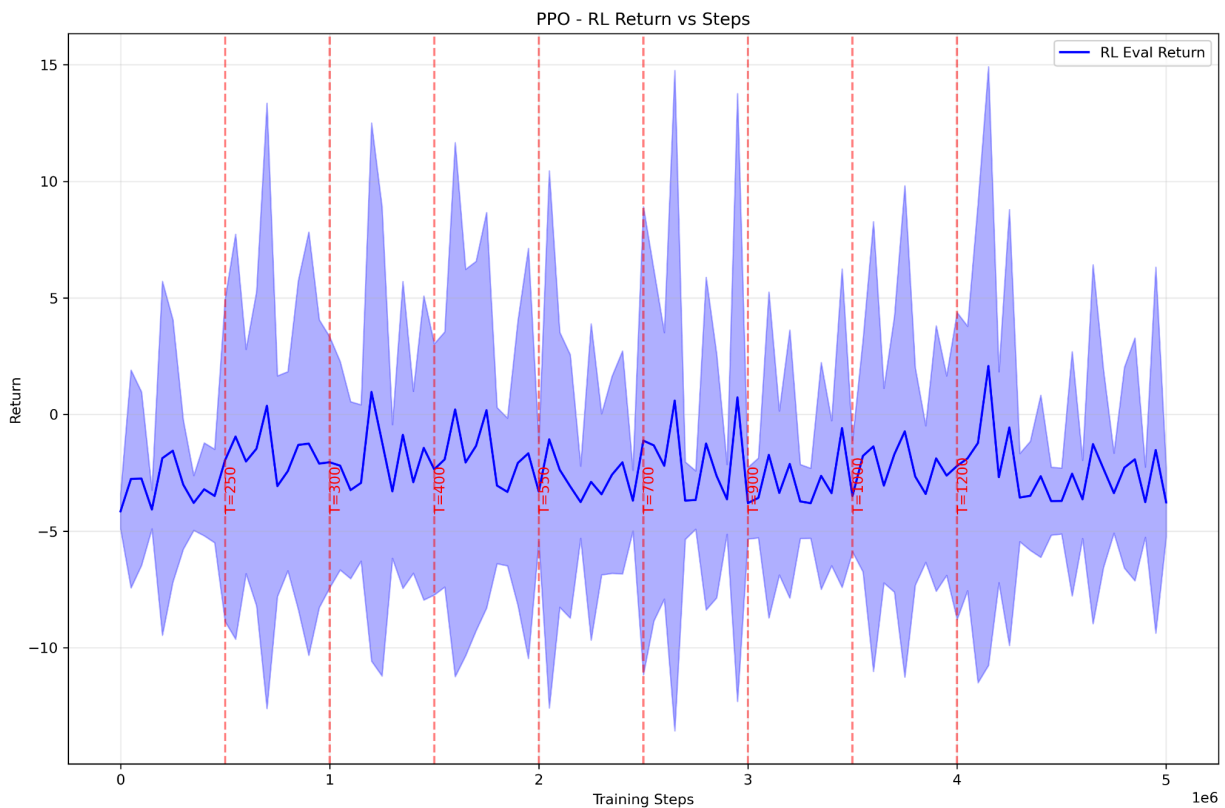
C. Parameter Tuning and Implementation Challenges

During development, many practical algorithm environment problems had to be resolved:

- Scroll ordering bugs caused the agent to clip into ground tiles
- Gravity calculation physics caused the player to fall to the bottom of the ground, dying instantly upon crossing a pit
- Tile overwriting issues, with fruit and obstacle generation overlapping, leading to misleading reward values for the model
- Reward shaping had to be tuned; e.g., I originally provided a reward for a safe landing, but it encouraged the agent to spam jump whenever it thought it was safe
- Normalization of certain distance features, which increased stability in training

III. RESULTS

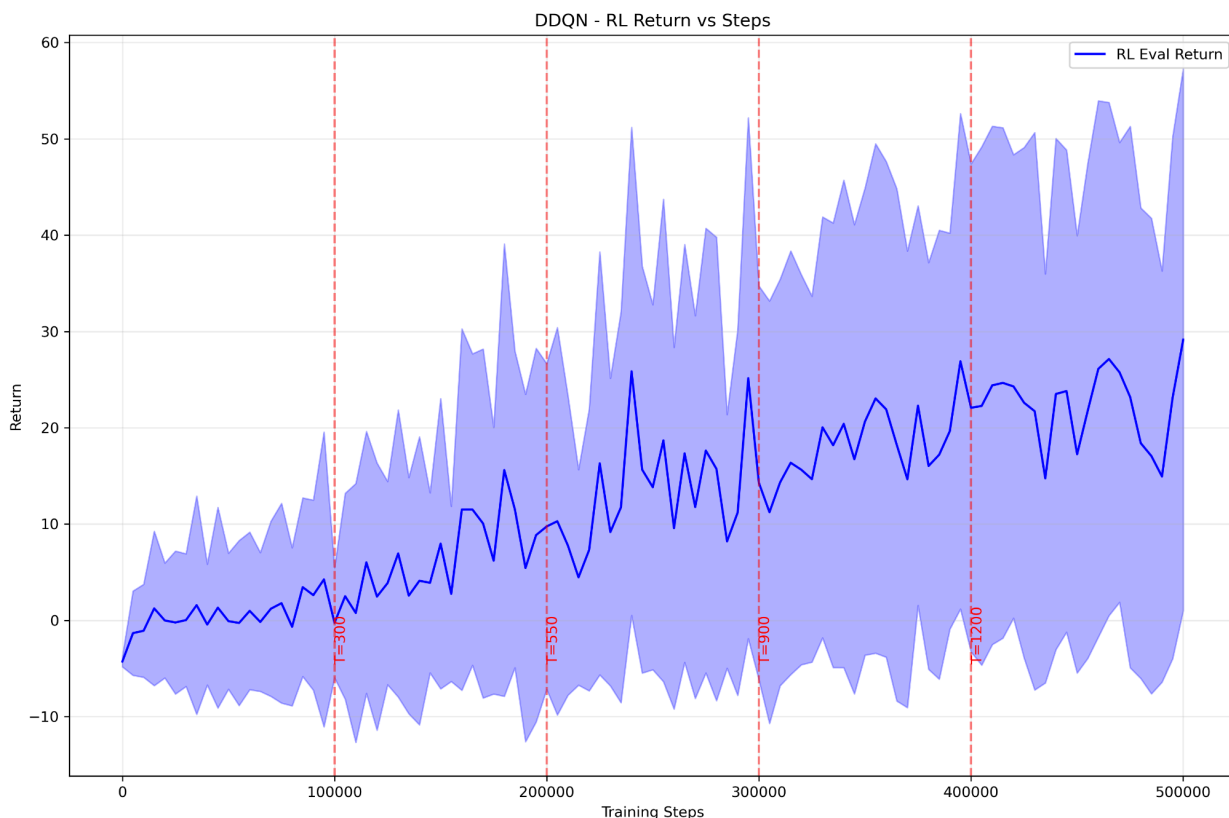
A. PPO Performance



The figure above shows the PPO agent's reinforcement learning evaluation results over 5 million training steps under the curriculum schedule. Despite heavy training and reward shaping, PPO shows very high variance and limited improvements across all time horizons. The mean hovers at around -3 to -1, occasionally obtaining positive returns. This matches observed gameplay behavior in the videos, where I saw that PPO can sometimes make good decisions and avoid obstacles, but fails to consistently perform well, often deciding to jump despite jumping leading to its future death from an airborne obstacle or pit.

Further examining the videos, we see that PPO tends to favor the jump actions, even deciding to jump when an obstacle above leads to death. This indicates that PPO seems to fail in terms of modelling these temporal dependencies, since the agent receives a sparse, noisy learning signal (due to randomness) rather than the dense, heavier environment PPO succeeds in. This is also why PPO struggles in similar sparse environments, with an example being MountainCar (which, when I tested PPO, failed to produce results, while it succeeded on LunarLander).

B. DDQN Performance



DDQN demonstrated a much better and more stable performance compared to PPO. As shown in the training curve above, we see that the average return steadily increased from the baseline of -4.5 (instant death) to around 20 over the 500,000 steps. Both algorithms took around 6 hours to train each, with DDQN training only a tenth of the number of steps PPO could train. Although variance remained high, reflecting occasional early death as well as rare successes, the upward trend shows that the agent was able to learn long horizon survival behaviors and decently effective jump timing.

IV. CODE STATUS & RESOURCES

The final code is fully functional, supporting training, evaluation, visualization, and working gameplay (code block right after JunimoKartEnv). Both PPO and DDQN agents can be trained on the environment and evaluated using the deterministic greedy policy, rendered frame

by frame to produce gameplay videos to visualize performance. The environment seems stable and successfully generates terrain and obstacles without runtime errors.

The experiments were produced on a single GPU using Pytorch and Gymnasium, with the PPO and DDQN implemented manually based on class material.

- Gymnasium Custom Environments - https://gymnasium.farama.org/introduction/create_custom_env/
- IPython for loading videos - <https://ipython.readthedocs.io/en/stable/api/generated/IPython.display.html#classes>
- Imageio for creating videos - <https://imageio.readthedocs.io/en/stable/examples.html>

V. CONCLUSION & FUTURE DIRECTIONS

This project showed that RL algorithms are able to learn in a custom designed platform environment, adapting to sparse rewards, procedural generation, and timing actions. Between the two methods, DDQN consistently learned a more stable and higher performance survival behavior than PPO, which struggled with jump timing, often deciding to jump into aerial obstacles or too often. These results show that the choice in algorithms matters strangely depending on the environment structure (as we often consider PPO better, but DDQN performed better in this case).

Future extensions could be exploring different algorithms other than the two I explored (SAC, DDPG, etc.), as well as implementing behavioral cloning or imitation learning. There is also room for exploration in terms of deepening environmental understanding, such as improvements to reward shaping, as well as hyperparameter sweeps to see how the algorithm changes. Lastly, you could also improve the environment, possibly allowing multiple levels with different mechanics, expanding the environment towards a fully fledged Junimo Kart clone.